

TCP (Transmission Control Protocol) is a **connection-oriented and reliable transport layer protocol**. It is used when data must reach the destination **accurately, completely, and in the correct order**. Before sending any data, TCP first establishes a connection between the client and the server using the **Three-Way Handshake (SYN, SYN-ACK, ACK)**. During communication, TCP uses **sequence numbers** and **acknowledgement numbers** to track data and confirm successful delivery. If any packet is lost, TCP retransmits it, making communication reliable. In this practical, we will use **Wireshark** to capture TCP packets while uploading a file from our computer to a remote server. Then we will analyze how TCP establishes the connection and transfers the data step by step.

Three-Way Handshake Process

1. **SYN (Synchronize):**
The **client** sends a **SYN** packet to the server requesting to establish a connection.
2. **SYN-ACK (Synchronize-Acknowledge):**
The **server** replies with a **SYN-ACK** packet, indicating that it has received the client's request and is ready to establish the connection.
3. **ACK (Acknowledge):**
The **client** sends an **ACK** packet to confirm the server's response. After this, the TCP connection is successfully established and data transfer begins.

Use of Sequence Number

- A **Sequence Number** is assigned to each TCP packet.
- It helps maintain the **correct order** of packets at the receiver.
- If a packet is lost, TCP can identify the missing packet and retransmit it.

Use of Acknowledgement Number

- An **Acknowledgement Number** is sent to give confirmation that packets have been received successfully.
- It indicates the **next sequence number expected** from the sender.
- If an acknowledgement is not received, TCP retransmits the missing packet, ensuring reliable communication.

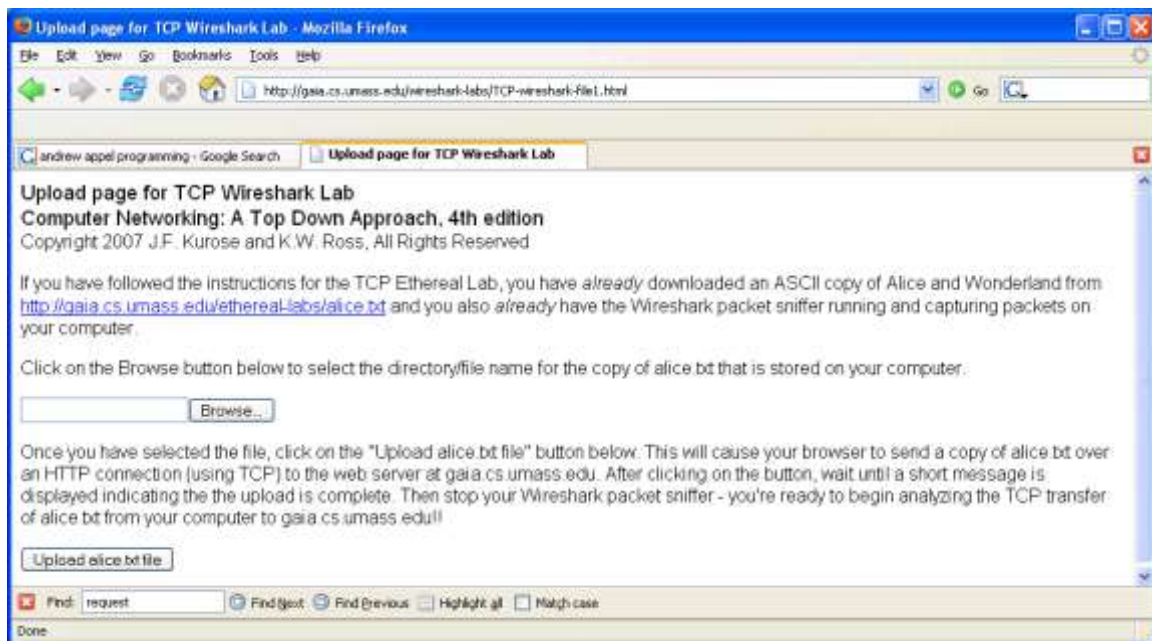
In this practical, we will study the working of the TCP (Transmission Control Protocol) using Wireshark. First, we will capture TCP packets while uploading a file from our computer to a remote server using the HTTP POST method. After capturing the packets, we will analyze different TCP features such as the TCP connection, Three-Way Handshake (SYN, SYN-ACK, ACK), sequence numbers, acknowledgement numbers, and data transfer. This practical helps us understand how TCP establishes a reliable connection and transfers data between the client and the server.

1. Capturing a bulk TCP transfer from your computer to a remote server

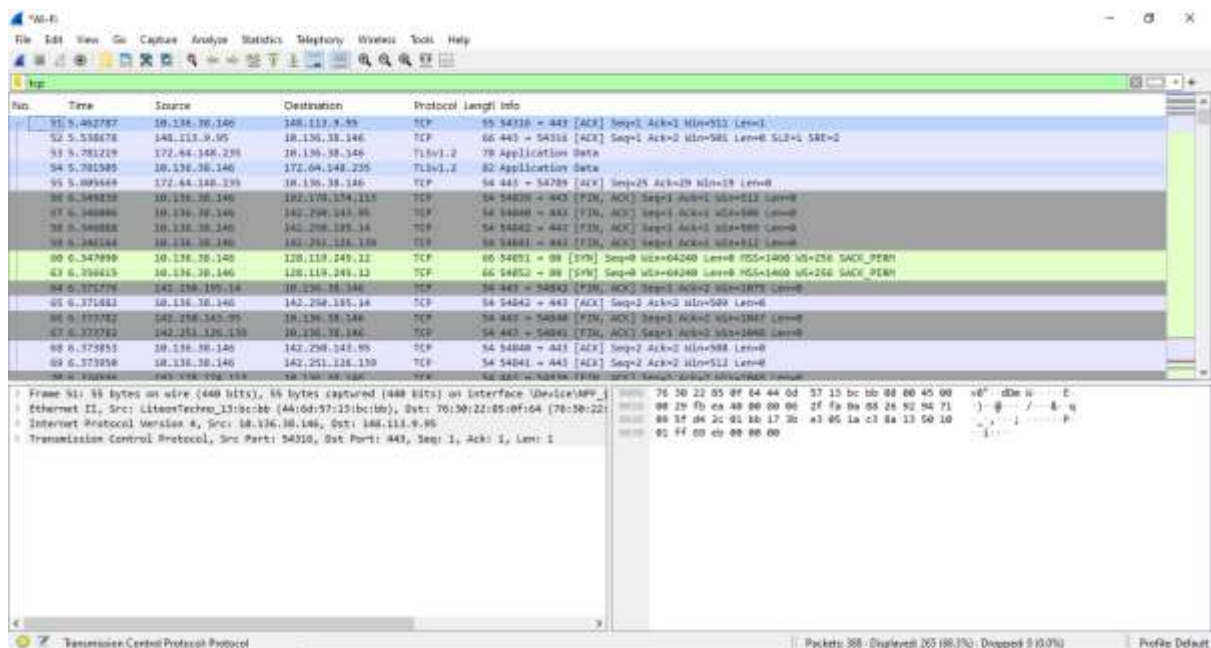
Before we start learning TCP, we need to capture TCP packets using Wireshark. For this, we will first download a file called `alice.txt`. This file contains the ASCII text of the book *Alice in Wonderland*. Then, we will open the given web page, select the `alice.txt` file from our computer, and upload it to a remote web server using the HTTP POST method. We use the POST method because it is used to send a large amount of data from the client to the server. While the file is being uploaded, Wireshark will capture all the TCP packets sent and received between our computer and the server. Later, we will analyze these captured TCP packets.

Do the following:

- Start up your web browser. Go the <http://gaia.cs.umass.edu/wireshark-labs/alice.txt> and retrieve an ASCII copy of *Alice in Wonderland*. Store this file somewhere on your computer.
- Next go to <http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html>



- Use the *Browse* button in this form to enter the name of the file (full path name) on your computer containing *Alice in Wonderland* (or do so manually). Don't yet press the "*Upload alice.txt file*" button.
- Now start up Wireshark and begin packet capture (*Capture->Start*) and then press *OK* on the Wireshark Packet Capture Options screen (we'll not need to select any options here).
- Returning to your browser, press the "*Upload alice.txt file*" button to upload the file to the gaia.cs.umass.edu server. Once the file has been uploaded, a short congratulations message will be displayed in your browser window.
- Stop Wireshark packet capture. Your Wireshark window should look similar to the window shown below.



Before analyzing the TCP connection in detail, let's first get an overview of the captured packets. First, apply the `tcp` filter in the Wireshark Display Filter bar (type `tcp` in lowercase and press Enter). This will display only TCP packets, making them easier to analyze.

Now check whether your capture contains these packets. First, look for the SYN packet, which indicates the start of the TCP connection (Three-Way Handshake). Then, look for the HTTP POST packet, which is used to upload the `alice.txt` file. If you can see these packets, it means the packet capture was successful.

Answer the question.

1. What is the IP address and TCP port number used by the client computer (source) that is transferring the file to gaia.cs.umass.edu?

To answer this question, the easiest way is to select the HTTP POST message (or any HTTP message of the same file upload) in Wireshark. Then, open the TCP packet details. In the packet details window, you can find the client's IP address and TCP port number.

Source ip address: 10.136.38.146

The image shows a Wireshark network traffic capture. The top pane displays a list of captured packets. Packet 302 is an HTTP POST request from 10.136.38.146 to 128.119.245.12. The middle pane shows the details of the selected packet (302), including the Ethernet II header, Internet Protocol Version 4 header, and TCP header. The bottom pane shows the raw packet data in hexadecimal and ASCII.

Packet 302: 955 bytes on wire (7640 bits), 955 bytes captured (7640 bits) on interface \Device\NPF{...}

Ethernet II, Src: LiteonTechno_13:bc:bb (44:6d:57:13:bc:bb), Dst: 76:30:22:85:0f:64 (76:30:22:85:0f:64)

Internet Protocol Version 4, Src: 10.136.38.146, Dst: 128.119.245.12

0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 941
Identification: 0xff0e (65422)
> 010. = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: TCP (6)
Header Checksum: 0x511e [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.136.38.146
Destination Address: 128.119.245.12
[Stream index: 10]

Frame (955 bytes) Reassembled TCP (153037 bytes)

Packets: 388 · Displayed: 265 (68.3%) · Dropped: 0 (0.0%) Profile: Default

Source port: 54851

Wireshark packet capture showing an HTTP POST request and its response. The packet list shows a POST request to /index.php from 10.136.38.146 to 128.119.245.12. The packet details show the request body as 'POST /index.php HTTP/1.1'. The packet bytes show the raw data of the request.

Frame 302: 955 bytes on wire (7640 bits), 955 bytes captured (7640 bits) on interface (b0c1bb) (44:6d:57:13:b0c1bb), Src: LiteonTechno... (44:6d:57:13:b0c1bb), Dst: 70:30:22:82:0f:04 (70:30:22:82:0f:04)

Ethernet II, Src: LiteonTechno... (44:6d:57:13:b0c1bb), Dst: 70:30:22:82:0f:04 (70:30:22:82:0f:04)

Internet Protocol Version 4, Src: 10.136.38.146, Dst: 128.119.245.12

Transmission Control Protocol, Src Port: 54851, Dst Port: 80, Seq: 152157, Ack: 1, Len: 981

Source Port: 54851

Destination Port: 80

[Stream Index: 6]

[Stream Packet Number: 149]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment (len: 981)]

Sequence Number: 152157 (relative sequence number)

Sequence Number (raw): 3854434663

[Next Sequence Number: 153838 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment Number (raw): 4000429734

Window ... = Header Length: 20 bytes (5)

Flags: RST (RST, ACK)

Frame 303: 955 bytes on wire (7640 bits), 955 bytes captured (7640 bits) on interface (b0c1bb) (44:6d:57:13:b0c1bb), Src: 70:30:22:82:0f:04 (70:30:22:82:0f:04), Dst: 10.136.38.146

Ethernet II, Src: 70:30:22:82:0f:04 (70:30:22:82:0f:04), Dst: 10.136.38.146

Internet Protocol Version 4, Src: 70.30.22.82, Dst: 10.136.38.146

Transmission Control Protocol, Src Port: 80, Dst Port: 54851, Seq: 1, Len: 0

Source Port: 80

Destination Port: 54851

[Stream Index: 6]

[Stream Packet Number: 150]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment (len: 0)]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 4000429734

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 153838 (relative ack number)

Acknowledgment Number (raw): 3854434663

Window ... = Header Length: 20 bytes (5)

Flags: RST (RST, ACK)

Frame 304: 955 bytes on wire (7640 bits), 955 bytes captured (7640 bits) on interface (b0c1bb) (44:6d:57:13:b0c1bb), Src: 70:30:22:82:0f:04 (70:30:22:82:0f:04), Dst: 10.136.38.146

Ethernet II, Src: 70:30:22:82:0f:04 (70:30:22:82:0f:04), Dst: 10.136.38.146

Internet Protocol Version 4, Src: 70.30.22.82, Dst: 10.136.38.146

Transmission Control Protocol, Src Port: 80, Dst Port: 54851, Seq: 1, Len: 0

Source Port: 80

Destination Port: 54851

[Stream Index: 6]

[Stream Packet Number: 151]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment (len: 0)]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 4000429734

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 153838 (relative ack number)

Acknowledgment Number (raw): 3854434663

Window ... = Header Length: 20 bytes (5)

Flags: RST (RST, ACK)

2. What is the IP address of gaia.cs.umass.edu? On what port number is it sending and receiving TCP segments for this connection?

Source ip: 128.119.245.12

The image shows a Wireshark packet capture of an HTTP GET request. The packet list on the left shows a packet from 128.119.245.12 to 10.136.38.146 on port 80. The packet details pane on the right shows the following information:

- Frame 341: 834 bytes on wire (6672 bits), 834 bytes captured (6672 bits) on interface libel
- Ethernet II, Src: 70:50:12:05:0f:04 (70:50:12:05:0f:04), Dst: LibenTechno_13:bc:bd (44:00:11:13:bc:bd)
- Internet Protocol Version 4, Src: 128.119.245.12, Dst: 10.136.38.146
- TCP, Src Port: 80, Dst Port: 80
- HTTP, GET /index.html HTTP/1.1

The packet bytes pane on the right shows the raw data of the packet, including the Ethernet II header, IP header, and TCP header.

Source port number:80

The image shows a Wireshark packet capture of a TCP segment. The packet list on the left shows a packet from 128.119.245.12 to 10.136.38.146 on port 80. The packet details pane on the right shows the following information:

- Frame 341: 834 bytes on wire (6672 bits), 834 bytes captured (6672 bits) on interface libel
- Ethernet II, Src: 70:50:12:05:0f:04 (70:50:12:05:0f:04), Dst: LibenTechno_13:bc:bd (44:00:11:13:bc:bd)
- Internet Protocol Version 4, Src: 128.119.245.12, Dst: 10.136.38.146
- Transmission Control Protocol, Src Port: 80, Dst Port: 80
- TCP, Seq: 35808, Len: 788

The packet bytes pane on the right shows the raw data of the packet, including the Ethernet II header, IP header, and TCP header.

3. What is the IP address and TCP port number used by your client computer (source) to transfer the file to gaia.cs.umass.edu?

4. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? What is it in the segment that identifies the segment as a SYN segment? (match these things for selecting a syn packet, these are present in http post packet)

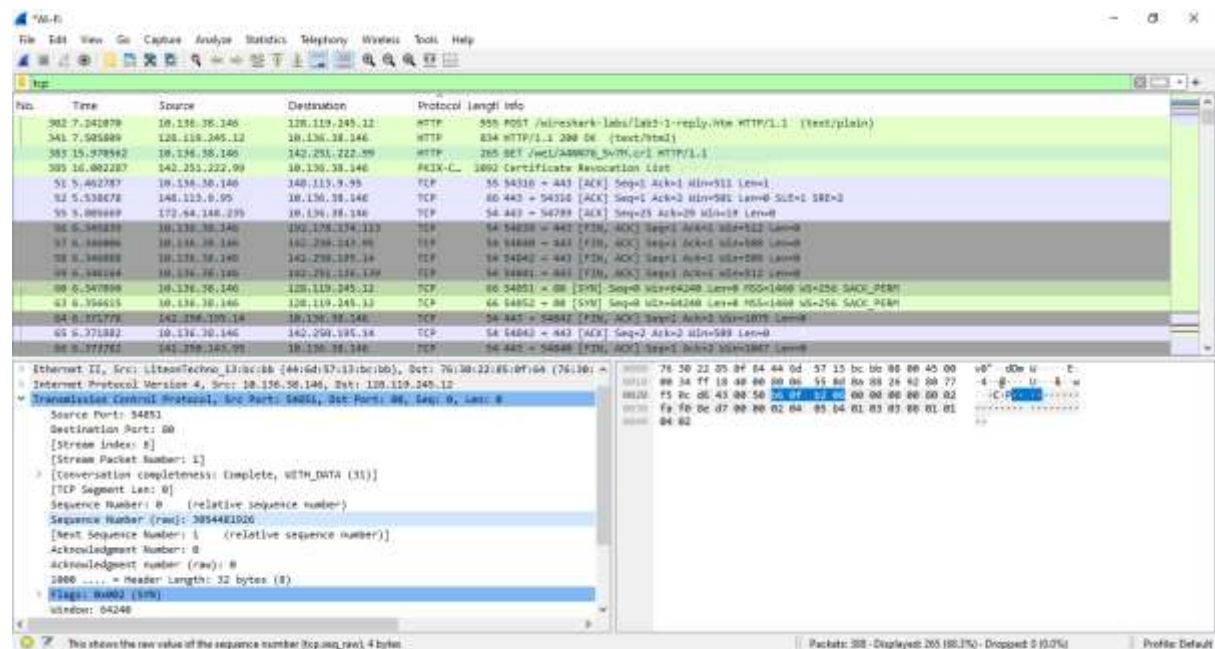
Ip of server: 128.119.245.12

```
source port :54851 (this is client port num)
```

Destination port :80 (this is server port num)

->sequence num:0

->Flags: 0x002 (SYN)



Wireshark interface showing a network capture. The top pane displays a list of packets, including HTTP requests and responses, and TCP segments. The middle pane shows the details of the selected packet (No. 1000), highlighting the TCP header fields. The bottom pane displays the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
382	7.241879	10.136.38.146	128.119.245.12	HTTP	355	POST /wirespark-labs/lab3-1-reply.htm HTTP/1.1 (text/plain)
383	7.505669	128.119.245.12	10.136.38.146	HTTP	834	HTTP/1.1 200 OK (text/html)
387	15.978942	10.136.38.146	142.251.222.99	HTTP	265	GET /url/348076_5v76/cr/ HTTP/1.1
395	16.982287	142.251.222.99	10.136.38.146	ACK	1002	Certificate Revocation List
51	5.462787	10.136.38.146	142.119.245.12	TCP	54	54310 → 443 [ACK] Seq=1 Ack=1 Win=512 Len=0
52	5.538078	142.119.245.12	10.136.38.146	TCP	60	443 → 54310 [ACK] Seq=1 Ack=1 Win=581 Len=0 SLS=1 SRE=2
55	5.889489	172.64.148.235	10.136.38.146	TCP	54	443 → 54789 [ACK] Seq=25 Ack=29 Win=65535 Len=0
60	6.348238	10.136.38.146	192.178.234.123	TCP	54	54838 → 443 [FIN, ACK] Seq=1 Ack=1 Win=512 Len=0
67	6.468864	10.136.38.146	142.251.222.99	TCP	54	54840 → 443 [FIN, ACK] Seq=1 Ack=1 Win=581 Len=0
68	6.348888	10.136.38.146	142.251.222.99	TCP	54	54842 → 443 [FIN, ACK] Seq=1 Ack=1 Win=581 Len=0
69	6.348144	10.136.38.146	192.178.234.123	TCP	54	54841 → 443 [FIN, ACK] Seq=1 Ack=1 Win=512 Len=0
88	6.547888	10.136.38.146	128.119.245.12	TCP	60	54851 → 88 [FIN] Seq=0 Win=64240 Len=0 RST=1460 WS=256 SACK_PERM
83	6.354415	10.136.38.146	128.119.245.12	TCP	66	54852 → 88 [FIN] Seq=0 Win=64240 Len=0 RST=1460 WS=256 SACK_PERM
84	6.375779	142.251.222.99	10.136.38.146	TCP	54	443 → 54842 [FIN, ACK] Seq=2 Ack=2 Win=581 Len=0
85	6.371882	10.136.38.146	142.251.222.99	TCP	54	54842 → 443 [ACK] Seq=2 Ack=2 Win=581 Len=0
86	6.377782	142.251.222.99	10.136.38.146	TCP	54	443 → 54848 [FIN, ACK] Seq=1 Ack=2 Win=581 Len=0

Details of packet 1000 (TCP segment):

- Acknowledgment number (raw): 0
- 1000 ... = Header Length: 32 bytes (8)
- Flags: 0x0002 (SYN)
- 0000 ... = Reserved: Not set
- ...0 ... = Accurate ECN: Not set
- ...0 ... = Congestion Window Reduced: Not set
- ...0 ... = ECN-ECN: Not set
- ...0 ... = Urgent: Not set
- ...0 ... = Acknowledgment: Not set
- ...0 ... = Push: Not set
- ...0 ... = Reset: Not set
- ...0 ... = SYN: Set
- ...0 ... = FIN: Not set
- [TCP Flags:S.]
- Window: 64240
- [Calculated window size: 64240]
- Checksum: 0x0007 [unverified]

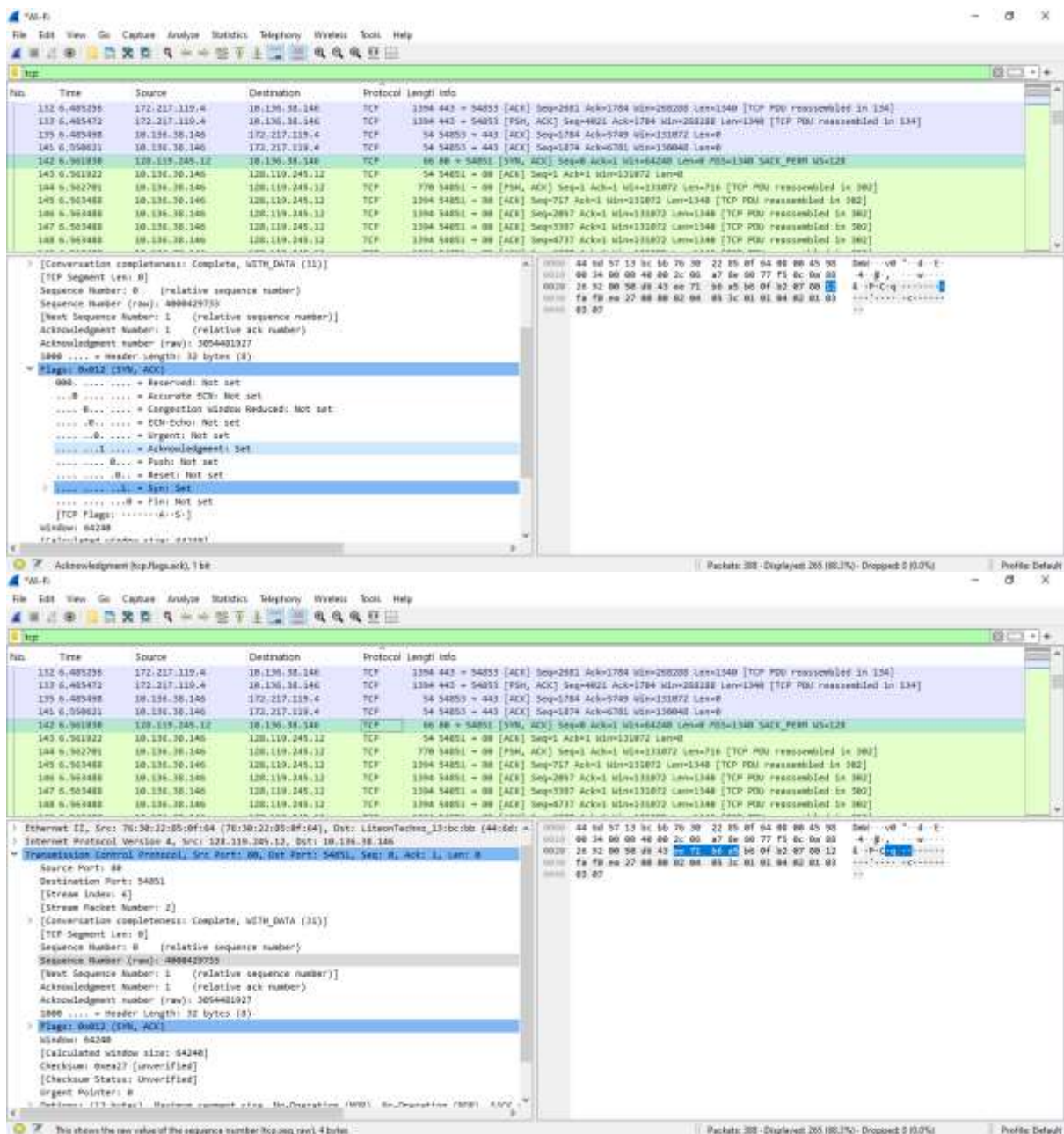
Raw packet data (hex):

```

0000  76 50 22 0f 64 44 0d 57 15 bc 0e 00 00 45 00  v0" d0w u ... E
0010  00 34 ff 18 48 80 00 00 55 80 00 00 28 42 00 77  -4-8...U--8-w
0020  f5 8c 0c 43 00 54 76 76 76 00 00 00 00 00 02  -C-P-6666.....
0030  fa 7b 0c d7 00 00 02 04 05 04 01 03 05 00 01 01  .....
0040  04 02
  
```

Summary: This shows the raw value of the sequence number (tcp.seq_raw), 4 bytes. Packets: 388 - Displayed: 265 (68.3%) - Dropped: 0 (0.0%) Profile: Default

5. What is the sequence number of the SYNACK segment sent by gaia.cs.umass.edu to the client computer in reply to the SYN? What is the value of the Acknowledgement field in the SYNACK segment? How did gaia.cs.umass.edu determine that value? What is it in the segment that identifies the segment as a SYNACK segment?



Sequence Number: 0 (relative). The Sequence Number of the SYN-ACK packet is 0.

Acknowledgement Number: 1 (relative). The Acknowledgement Number of the SYN-ACK packet is 1.

ACK = 1 because the client sent a SYN packet with Sequence Number 0. The server acknowledges the next expected Sequence Number, so it sends ACK = 1.

The packet is identified as a SYN-ACK packet because the TCP Flags field contains both SYN and ACK flags (Flags = SYN, ACK / 0x012).

6. Find the TCP packet that contains the HTTP POST command. What is its Sequence Number?

To find the HTTP POST command, find the packet whose Data field contains the word "POST".

The image shows a Wireshark packet capture of an HTTP session. The packet list on the left shows several packets, with packet 302 highlighted. The packet details pane on the right shows the structure of the selected packet (Frame 302: 955 bytes on wire (7640 bits), 955 bytes captured (7640 bits) on interface \Device\NPF...).

The packet details pane shows the following information:

- Ethernet II, Src: Liteasfetho013bc1bb (AA:00:07:13:bc:1bb), Dst: 76:08:22:85:0f:64 (76:08:22:85:0f:64)
- Internet Protocol Version 4, Src: 10.136.38.146, Dst: 128.119.245.12
- Transmission Control Protocol, Src Port: 54851, Dst Port: 80, Seq: 352137, Ack: 1, Len: 955
- TCP Segment Len: 955
- Sequence Number (raw): 352137 (relative sequence number)
- Stream Sequence Number: 135838 (relative sequence number)
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 4880429734
- 0000 = Header Length: 20 bytes (5)
- Flags: 0018 (PSH, ACK)
- 0000 = Reserved: Not set
- ...0 = Accurate ECN: Not set
-0.... = Congestion Window Reduced: Not set
-0.... = ECU-Echo: Not set

The packet bytes pane on the right shows the raw data of the packet, with the word "POST" visible in the data field.

(use any ack packet after connection)

